

Yes. Based on everything we've done throughout the project, here is a comprehensive report covering the **MISP (Malware Information Sharing Platform)** implementation. This report focuses on the actual implementation, architecture, workflow, integrations, detection engineering, findings, and important technical facts from our project, excluding unrelated components.

MISP Implementation Report

Hybrid Enterprise SOC Project

1. Objective

The objective of integrating MISP into our Hybrid Enterprise SOC was to provide centralized Threat Intelligence capable of automatically collecting, enriching, classifying and distributing Indicators of Compromise (IOCs) to the SOC platform.

Instead of manually maintaining malicious IPs, domains, hashes and URLs, MISP continuously synchronizes intelligence feeds and makes them available for detection inside Elastic SIEM.

2. MISP Architecture

Threat Feeds

|



MISP

|

| REST API



Python Collector

|



Kafka Producer

|



Kafka

|



Logstash Consumer

|



Elasticsearch Indexes

|



Elastic Detection Rules

|



Security Alerts

The complete architecture separates threat intelligence collection from SIEM ingestion, making the pipeline modular and scalable.

3. Major Components

MISP Server

Central Threat Intelligence Platform

Responsibilities

- Stores threat events
- Stores IOC attributes
- Provides REST API
- Synchronizes threat feeds
- Classifies events
- Maintains threat metadata

Python Threat Collector

Custom Python application responsible for communicating with MISP.

Functions

- Authenticate using API Key
- Download MISP events
- Parse JSON
- Normalize fields
- Publish messages into Kafka

Apache Kafka

Acts as a message broker between MISP and Logstash.

Advantages

- Decouples producer and consumer
- Prevents data loss
- Buffers incoming intelligence
- Supports scalable ingestion

Logstash

Consumes Kafka topics.

Responsibilities

- Parse JSON
- Clean fields
- Normalize data
- Add metadata
- Send documents into Elasticsearch

Elasticsearch

Stores all intelligence as searchable documents.

Indexes created

misp-threat-intelligence

misp-ioc-intelligence

Kibana Security

Uses the indexed intelligence for

- Searching
- Dashboards
- Detection Rules
- Alerts

- Investigation

4. MISP Docker Deployment

MISP was deployed as a containerized service.

Benefits

- Easy deployment
- Portable
- Isolated environment
- Simple upgrades
- API availability

Docker networking allowed communication with

- Kafka
- Logstash
- Elasticsearch

5. Threat Intelligence Workflow

Threat Feed

↓

MISP imports events

↓

Python fetches events

↓

Kafka Topic

↓

Logstash



Elasticsearch



Elastic Security Rule



SOC Alert

6. Elasticsearch Indices

Two major indices were created.

A. misp-threat-intelligence

Contains complete MISP Events.

Example fields

info

source

threat_level_id

publish_timestamp

@timestamp

error.message

This index was mainly used for event-level detection.

B. misp-ioc-intelligence

Stores Indicators of Compromise.

Contains

- IP addresses
- Domains
- URLs
- File hashes
- Email addresses

This index is intended for IOC matching and enrichment. During the project, IOC correlation rules were intentionally deferred because they require endpoint or network telemetry capable of generating IOC matches.

7. Threat Intelligence Fields Used

During implementation we inspected mappings and verified usable fields before building rules.

Important fields included:

threat_level_id

info

source

publish_timestamp

@timestamp

error.message

We queried Elasticsearch to verify field availability before creating detection logic.

8. Threat Levels

MISP uses four threat levels.

ID Meaning

- 1 High / Critical Threat
- 2 Medium Threat
- 3 Low Threat
- 4 Undefined / Unknown

Threat level formed the basis for rule severity.

9. Detection Rules Implemented

Instead of relying on prebuilt content, custom Elastic Query rules were developed against the MISP indices.

Implemented rules included:

Critical MISP Threat Intelligence

Purpose

Alert whenever a critical (Threat Level 1) event exists.

Example logic

threat_level_id : 1

AND source : "MISP"

Severity

High

Published Critical MISP Threat

Purpose

Alert only for newly published critical intelligence.

Final implementation relied on event ingestion time to avoid repeated alerts from historical data.

Logic included:

- Threat Level 1
- MISP source
- Recent ingestion window using @timestamp
- Exclusion of parser errors

Severity

Critical

New MISP Threat Intelligence

Purpose

Detect newly ingested MISP intelligence regardless of criticality within the configured severity level.

Severity

Medium

10. Rule Engineering Findings

Several important observations emerged while designing these rules:

- Detection rules require understanding of the actual indexed fields before writing KQL queries.
 - Directly filtering on static fields such as `publish_timestamp` was ineffective because the field was mapped as text.
 - Using `@timestamp` (ingestion time) provided reliable detection of newly imported events.
 - Adding source : "MISP" prevented unintended matches from unrelated data.
 - Excluding `error.message` helped avoid generating alerts for parsing failures.
-

11. Detection Workflow

MISP Event

↓

Logstash

↓

Elasticsearch

↓

Rule Scheduler



Elastic Detection Engine



Alert



SOC Investigation

12. Alert Metadata

Rules were configured with:

- Severity
- Risk Score
- Tags
- Schedule
- Look-back Window
- Index Pattern
- Custom Query

Alerts appeared inside

Security



Alerts

13. Scheduling

Detection rules executed periodically with a defined interval and additional look-back time to ensure events were not missed due to ingestion delays.

14. Threat Intelligence Dashboard

Elastic Security displayed

- Total Alerts
- Alert Severity
- Rule Distribution
- Timeline
- Investigation View

Implemented categories included:

- Critical Threats
 - Published Threats
 - New Threat Intelligence
-

15. Validation Process

Implementation was validated by:

- Querying Elasticsearch directly
 - Verifying index mappings
 - Inspecting sample documents
 - Confirming rule execution
 - Monitoring generated alerts
 - Reviewing Elastic Security dashboards
-

16. Important Technical Findings

Field Verification

Detection logic should always be based on actual Elasticsearch mappings rather than assumptions.

Static vs Dynamic Fields

Static publication timestamps do not reliably indicate newly ingested intelligence.

Using ingestion time (@timestamp) is more suitable for "new intelligence" detection.

Threat Categorization

threat_level_id provides a simple and effective mechanism for assigning detection severity.

Rule Scope

Using a source filter (source : "MISP") narrows detection to threat intelligence data and reduces false positives.

Parser Error Exclusion

Ignoring documents with error.message prevents alerts on malformed or failed ingestions.

17. Limitations Identified

- IOC correlation rules were intentionally not implemented because the environment lacked endpoint/network telemetry capable of producing IOC match events.
 - Suricata-based threat intelligence correlation was postponed due to limited validation and implementation knowledge at that stage.
 - The publish_timestamp field was stored as text, limiting direct time-based filtering.
 - High alert volumes from historical intelligence required refining rules to focus on newly ingested events.
-

18. Technologies Used

- MISP
- Docker
- Python
- Kafka
- Logstash
- Elasticsearch
- Kibana
- Elastic Security
- REST API
- JSON
- KQL (Kibana Query Language)

19. Overall Outcome

The project successfully established an end-to-end threat intelligence pipeline where MISP collected external intelligence, a custom Python service retrieved events through the MISP REST API, Kafka buffered the data, Logstash normalized and indexed it into Elasticsearch, and Elastic Security generated actionable alerts using custom detection rules. This demonstrated automated threat intelligence ingestion, event classification, and SOC alert generation, while laying the foundation for future IOC correlation once endpoint or network telemetry (such as Suricata or Elastic Agent) is available.